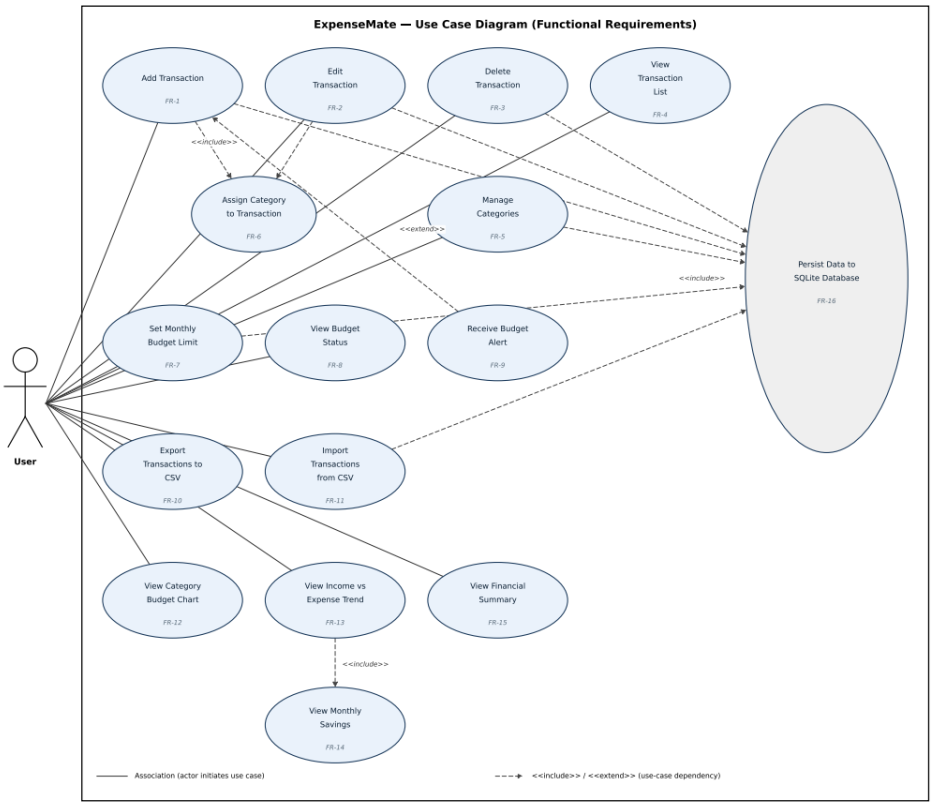
Expense Mate
Members: Areesha 24k-3043 , Burhan 24k-3056
Phase 01
Software Requirements Specification (SRS)

1. System Scope and Objectives

ExpenseMate is a standalone personal expense manager running on localhost. The system allows users to track income and expenditure, manage custom categories, define monthly budget limits, receive real-time threshold warnings, visualize multi-series savings trends, and back up transactions via CSV. All data persists in a local SQLite database.

2. Use Case Model and Traceability



Use Case ID	Use Case Name	Actor	Included / Extended Dependencies	FR Ref
UC-01	Add Transaction	User	Includes UC-06 (Assign Category), UC-16 (Persist to DB)	FR-1
UC-02	Edit Transaction	User	Includes UC-06 (Assign Category), UC-16 (Persist to DB)	FR-2
UC-03	Delete Transaction	User	Includes UC-16 (Persist to DB)	FR-3
UC-04	View Transaction List	User	Queries local repository with Newest/Oldest sort	FR-4
UC-05	Manage Categories	User	Extends UC-06 (Creates custom category pool)	FR-5
UC-06	Assign Category	System	Enforces 1:1 foreign key binding per transaction	FR-6
UC-07	Set Monthly Budget Limit	User	Includes UC-16 (Persists standing monthly limit)	FR-7
UC-08	View Budget Status	User	Calculates remaining = limit - expenses (current month)	FR-8
UC-09	Receive Budget Alert	User	Triggered when category expenses strictly exceed budget limit	FR-9
UC-10	Export to CSV	User	Exports date,category,amount,type to UTF-8 file	FR-10
UC-11	Import from CSV	User	Includes UC-16 (All-or-nothing batch persistence)	FR-11
UC-12	View Budget Chart	User	Plots multi-bar chart: Budget, Spent, Remaining by category	FR-12
UC-13	View Income vs Expense	User	Includes UC-14 (View Monthly Savings series)	FR-13
UC-14	View Monthly Savings	User	Calculates Net = Income - Expenses (supports negative values)	FR-14
UC-15	View Financial Summary	User	Summarizes Income, Expenses, Net for chosen date range	FR-15
UC-16	Persist Data to SQLite	Database	Atomic SQL operations enforcing relational integrity	FR-16

3. Functional Requirements (FR) Specification

ID	Subsystem	Requirement Description
FR-1	Transactions	The system shall allow adding a transaction with amount, category, date, and type (income/expense).
FR-2	Transactions	The system shall allow editing all four fields of an existing transaction.
FR-3	Transactions	The system shall allow deleting an existing transaction after explicit user confirmation.
FR-4	Transactions	The system shall display a sortable tabular list of transactions showing date, amount, category, and type.
FR-5	Categories	The system shall allow creating unique, custom categories (Food, Rent, Transport seeded by default).
FR-6	Categories	The system shall enforce that each transaction is assigned to exactly one existing category.
FR-7	Budgets	The system shall allow setting a standing monthly expenditure limit per category.
FR-8	Budgets	The system shall calculate the remaining budget (limit - expenses) for each category in the current month.
FR-9	Budgets	The system shall alert the user immediately when spending in a category strictly exceeds its budget.
FR-10	Portability	The system shall allow exporting all stored transactions to a CSV file.
FR-11	Portability	The system shall allow importing transactions from CSV under an all-or-nothing validation rule.
FR-12	Analytics	The system shall display bar charts showing budget, spent, and remaining balance by category.
FR-13	Analytics	The system shall render a monthly bar chart comparing income versus expenses over time.
FR-14	Analytics	The system shall display monthly net savings (income - expenses) as a third series on the trend chart.
FR-15	Analytics	The system shall display total income, expenses, and net balance metrics for any chosen date range.
FR-16	Persistence	The system shall persist all transactions, categories, and budgets in an embedded SQLite database.

4. Non-Functional Requirements (NFR) and Quality Attributes

ID	Attribute	Requirement Specification
NFR-1	Performance	Transaction list displays within 2.0 seconds for up to 500 stored records.
NFR-2	Usability	Clear input labels, placeholder hints, and help text for all transaction input fields.
NFR-3	Feedback	Unambiguous success flash banners and explicit validation error alerts displayed on submission.
NFR-4	Reliability	Full persistence of state in local SQLite database ensuring zero data loss across application restarts.
NFR-5	Integrity	Incomplete, negative, or malformed transaction data is blocked from saving to the database.
NFR-6	CSV Integrity	CSV files missing required columns or containing invalid rows are rejected with atomic rollback.
NFR-7	Architecture	Three-tier modular separation: Presentation (UI), Business Logic (Services), and Persistence (Data).
NFR-8	Testing	Automated unit test coverage for budget mathematics, currency parsing, and CSV validation routines.
NFR-9	Validation	Numeric validation rejecting negative numbers, zero limits, and non-numeric inputs.
NFR-10	Consistency	Uniform layout, typography, primary buttons, and metric cards across all application pages.
NFR-11	Dates	Strict ISO YYYY-MM-DD calendar validation between 1900 and 2100 (rejecting Feb 30, etc.).
NFR-12	Fault Handling	Database exceptions are caught gracefully and shown to the user without crashing the process.
NFR-13	Latency	User mutations (Add/Edit/Delete) respond within 2.0 seconds under normal operation.
NFR-14	Resources	Runs efficiently on standard student hardware using local Python runtime and SQLite engine.
NFR-15	Consistency	Budget metrics and status cards immediately synchronize and update following any transaction mutation.

5. Baseline Architectural and Business Constraints

1. Categories: Food, Rent, Transport, Bills, Entertainment, Other pre-seeded. Custom categories supported (FR-5).
2. Budget Scope: Standing monthly limit evaluated against current calendar month. Only expenses decrement budgets.
3. CSV Schema: Strict column order (date, category, amount, type); unknown categories rejected (no automatic category creation).